

Chapter 4

Building Business Logic with Apex

Sprint 4 – Giving Intelligence to the Placement Management System

“Software begins to create value when it starts making correct business decisions consistently.”

Learning Outcomes

By the end of this sprint, you should be able to:

- Understand why enterprise applications require business logic.
 - Distinguish between data and business decisions.
 - Identify business rules from customer requirements.
 - Design Apex classes based on business responsibilities.
 - Explain why good engineers spend time understanding problems before writing code.
 - Think like a consultant while analysing requirements.
 - Prepare to implement business logic using Apex.
-

Welcome to Sprint 4

Good morning, Developer.

Over the past few days, your team has built the foundation of the Placement Management System. The application now stores students, companies, job opportunities and applications. Information is flowing into Salesforce, and the Placement Office is excited to see the progress.

The user interface looks professional. Records can be created, viewed and updated. Everyone appreciates how much has been accomplished.

At first glance, the project appears to be complete.

However, this morning's Sprint Review reveals a very different reality.

The Placement Officer enters the meeting room carrying a small stack of printed reports.

She smiles at the development team.

"Your application looks excellent."

Everyone smiles back.

Then she places the reports on the table.

"Unfortunately, I have a few concerns."

She opens the first report.

"Yesterday, a student with six active backlogs applied for a company that clearly specified 'No Active Backlogs'. The application was accepted."

She opens another report.

"A student submitted four applications for the same job."

Another report.

"Applications were accepted even after the deadline."

Another report.

"A company accidentally created duplicate job postings."

She looks around the room.

"The application stores all the information I enter."

She pauses for a moment.

"But it doesn't help me manage placements."

Silence fills the room.

Your Technical Lead walks to the whiteboard and writes just one sentence.

The system stores data, but it does not yet understand the business.

He turns towards the team.

"Until now we have been building a database."

Another pause.

“Today we begin building software.”

Welcome to Sprint 4.

Today, you will give intelligence to the Placement Management System.

Today's Engineering Principle

Many beginners believe that software development is about writing code.

Professional software engineers know something very different.

They know that programming languages are only tools.

The real objective is to solve business problems.

Customers never purchase software because it contains thousands of lines of code.

They purchase software because it consistently performs the right actions at the right time.

Every line of code that you write should answer one simple question:

Which business problem does this solve?

If a developer cannot answer that question, the code probably should not exist.

Throughout your career, technologies will change.

Programming languages will evolve.

Frameworks will come and go.

However, one principle will remain unchanged.

Understand the business first. Write the code afterwards.

If you cultivate this habit early in your career, every technology that you learn later will become easier to understand.

4.1 What Makes Software Intelligent?

Imagine a calculator.

You enter two numbers.

It performs an addition and displays the result.

The calculator follows a rule.

Now imagine the Placement Management System.

A student applies for a company.

Should the application always be accepted?

Certainly not.

The software must first answer several questions.

- Does the student satisfy the minimum CGPA requirement?
- Does the student have active backlogs?
- Has the application deadline expired?
- Has the student already applied for this job?
- Is the student already holding another offer that prevents further applications?
- Has the company restricted applications to certain branches?

Only after evaluating these conditions should the software decide whether to accept or reject the application.

This decision-making ability is called **business logic**.

Business logic is the intelligence that allows software to behave according to the needs of an organisation.

Without business logic, software becomes nothing more than an electronic register.

It stores information.

It does not assist people in making decisions.

Think Like an Engineer

Pod Discussion (20 Minutes)

Do not open your laptop.

Discuss the following questions with your pod members.

Write your answers in your engineering journal.

Question 1

Why should software reject an application instead of allowing the Placement Officer to reject it later?

Ans : It saves time and ensures only valid applications are submitted.

Question 2

What problems might occur if every student is allowed to apply for every company?

Ans : It can create unnecessary applications and increase the Placement Officer's workload.

Question 3

Which decisions should always be made automatically?

Which decisions should remain with the Placement Officer?

Discuss your reasons.

Ans : Eligibility and duplicate checks should be automatic. Final selection and approval should be handled by the Placement Officer.

Question 4

Can you think of five additional business rules that may be required in the future?

Remember that businesses evolve continuously.

Your software should be prepared for change.

Ans :

1. Limit the number of applications per student.
2. Check the application deadline.
3. Prevent students with backlogs from applying.
4. Allow only eligible branches to apply.
5. Block applications after a student is already selected.

Engineering Insight

Notice something interesting.

You have already begun designing software.

Yet you have not written a single line of Apex.

This is exactly how enterprise software projects begin.

The quality of the final solution depends far more on the quality of the questions asked than on the speed of coding.

4.2 Understanding Business Rules

Every organisation has rules.

Schools have attendance rules.

Banks have transaction rules.

Hospitals have patient care rules.

Universities have examination rules.

Similarly, the Placement Office also operates according to a well-defined set of rules.

Some examples are shown below.

Business Requirement	Business Rule
Students should not apply after the deadline.	Reject late applications.
Duplicate applications are not allowed.	Check existing applications before saving.
Companies specify eligibility criteria.	Validate CGPA, branch and backlog requirements.
Recruiters should know when eligible students apply.	Send notifications after successful submission.
Placement Officers should maintain accurate data.	Prevent duplicate company records where appropriate.

Notice that none of these statements mention Apex.

They simply describe how the business wants the software to behave.

Only after these rules are understood can developers decide how to implement them.

This is why professional software engineers spend considerable time discussing requirements before writing code.

Tech Lead's Advice

“Never ask, ‘What code should I write?’

Ask instead, ‘What decision should the software make?’

Good code is only the consequence of good thinking.”

4.3 From Business Responsibilities to Apex Classes

The Technical Lead draws two columns on the whiteboard.

On the left he writes:

Placement Office Responsibilities

- Register students
- Publish jobs
- Receive applications
- Check eligibility
- Notify recruiters
- Generate reports

On the right he leaves the space blank.

He turns towards the team.

“Every responsibility on the left must eventually become software.”

The room becomes quiet.

Then he continues.

“Software is easier to build when responsibilities are organised.”

He draws another box.

Student Service

“This class is responsible for student-related operations.”

Another box.

Job Service

“This class manages job-related operations.”

Another.

Application Service

“This class handles application processing.”

Suddenly the class begins to understand something important.

Classes are not created because Apex requires them.

Classes exist because businesses have responsibilities that must be organised carefully.

Whenever you are unsure whether a new class is required, ask yourself one question.

Is there a new business responsibility that deserves its own place?

If the answer is yes, you probably need a new class.

If the answer is no, you should reconsider your design.